

Bộ câu hỏi & trả lời — Data & Tiền xử lý (ôn trước khi trình bày)

Dùng để **ôn hiểu**, không học thuộc lòng — khi trả lời (nói hoặc viết tay) hãy diễn đạt bằng lời của mình. Mỗi câu trả lời cố tình viết ngắn, gạch đầu dòng, đúng kiểu cần khi viết tay trong 15 phút. Bám sát đúng mã nguồn trong `rba_local_project/src/` — không có ý nào bịa thêm ngoài code. Mỗi câu đều có dòng ► **Code chỉ rõ hàm nào, nằm ở file nào** — để nếu cô hỏi thêm "chỗ đó em code ở đâu?" thì mở đúng file ngay. Bản giải thích dài, chi tiết hơn (có ví dụ tính tay + số liệu thật): xem `GiaiThich_Data_TienXuLy.md` (hoặc bản PDF cùng tên).

Bản đồ file — nhớ 4 file cốt lõi theo đúng luồng chạy

Thứ tự	File	Nhiệm vụ	Hàm chính
1	<code>features.py</code>	Sinh đặc trưng	<code>load_raw()</code> , <code>engineer_features()</code> , <code>fit_global_stats()</code> , <code>apply_global_stats()</code>
2	<code>fuzzy_mamdani.py</code>	Hệ suy diễn mờ	<code>triangular()</code> , <code>low_med_high()</code> , lớp <code>MamdaniFuzzyRiskSystem</code>
3	<code>dataset_prep.py</code>	Chia tập + fit tiền xử lý	<code>prepare_splits()</code>
4	<code>model.py</code>	Mạng MLP + huấn luyện/đánh giá	<code>MLPClassifier</code> , <code>train_mlp()</code> , <code>evaluate()</code>

Phụ: `config.py` (hằng số, **không có hàm**) · `train.py` (`main()` — script chính) · `inference.py` (`predict_risk()`) · `visualize.py` (các hàm `plot_*`).

Nhóm 1 — Câu hỏi cơ bản về dữ liệu

Q1. Bộ dữ liệu lấy từ đâu, có gì đặc biệt?

- Nguồn: *Login Data Set for Risk-Based Authentication* (Wiefling và cộng sự) — log đăng nhập thật đã ẩn danh hóa.
- ~500.000 dòng, mỗi dòng = 1 lượt đăng nhập.
- Đặc biệt: chỉ ~3% là **tấn công** → mất cân bằng lớp nặng.
- ► **Code**: đường dẫn `DATA_PATH`, `DATA_FILENAME` khai báo ở `config.py`; đọc file bằng `load_raw()` — `features.py`.

Q2. Vì sao chọn nhãn `Is Attack IP` mà không phải nhãn khác?

- `Is Account Takeover`: chỉ ~4 mẫu dương / 500.000 dòng → quá ít để học.
- `Login Successful`: phản ánh lỗi kỹ thuật (sai mật khẩu, mạng lỗi...), không phải rủi ro bảo mật.
- `Is Attack IP`: ~3% dương — đủ dữ liệu để học, và đúng bản chất bài toán "chấm điểm rủi ro".
- ► **Code**: `TARGET_COLUMN = "Is Attack IP"` ở `config.py`.

Q3. Vì sao không dùng Accuracy để đánh giá mô hình?

- Chỉ cần đoán "không phải tấn công" cho **mọi** dòng đã đạt Accuracy ~97% — vô nghĩa, không phát hiện được tấn công nào.
- Dùng AUPRC/Recall/Precision — các chỉ số nhạy với lớp thiểu số.

- Lúc train cũng phải bù lệch lớp bằng `pos_weight` trong hàm loss.
 - ► **Code:** các chỉ số tính ở `evaluate()` — `model.py`; `pos_weight` đặt trong `train_mlp()` — cũng `model.py`.
-

Nhóm 2 — Feature engineering (sinh đặc trưng)

Q4. Từ dữ liệu thô sinh ra bao nhiêu đặc trưng, chia nhóm gì?

- Tổng **19 đặc trưng số** + 1 cột phân loại (`Device Type`, sau đó one-hot thành 4 cột).
- **4 nhóm** (đúng như 4 khối comment trong code):
 1. **Thời gian** — 4 đặc trưng, tính trực tiếp từ timestamp.
 2. **Mạng (RTT)** — 2 đặc trưng, có xử lý giá trị thiếu.
 3. **Độ hiếm Country/ASN** — 2 đặc trưng, cần thống kê từ tập train.
 4. **Hành vi lịch sử user** — 11 đặc trưng, dùng `shift/cumsum` theo từng user.
- Kiểm tra lại: $4 + 2 + 2 + 11 = 19$ [OK]
- ► **Code:** `engineer_features()` — `features.py` (danh sách 19 cột ở hằng số `FEATURE_COLUMNS_NUMERIC`, cùng file). Riêng nhóm 3 và `rtt_filled` tạo ở `apply_global_stats()` — cũng `features.py`.

Q5. RTT bị thiếu thì xử lý thế nào, sao không bỏ luôn dòng đó?

- Không bỏ dòng, vì mất dữ liệu và việc "thiếu RTT" tự nó **có thể là 1 tín hiệu** (liên quan loại mạng/thiết bị).
- Cách xử lý: tạo cột `rtt_missing` (an toàn, chỉ nhìn đúng dòng đó); giá trị thực tế điền bằng **median của tập train** (`rtt_filled = 535,0 ms`) — không phải median của cả tập.
- Dùng median chứ không dùng mean vì RTT lệch, có nhiều giá trị ngoại lai lớn.
- ► **Code:** `rtt_missing` ở `engineer_features()`; `rtt_filled` ở `apply_global_stats()` — cả hai trong `features.py`.

Q6. `is_new_country` (và các `is_new_*` khác) tính như thế nào?

- So sánh giá trị hiện tại với giá trị của **lần đăng nhập ngay trước đó** (`shift(1)`) của cùng user.
 - Khác → 1, giống → 0. Lần đăng nhập đầu tiên của user → quy ước tất cả = 1 (vì mọi thứ đều "mới" với họ).
 - 6 cột `is_new_*` cộng lại thành `num_changes` (0–6): đổi 1 thứ là bình thường, đổi cả 6 thứ thì gần như không còn là người dùng đó.
 - ► **Code:** `engineer_features()` — `features.py` (`shift` là hàm của `pandas`).
-

Nhóm 3 — Chống rò rỉ dữ liệu (trọng tâm, cô hay hỏi chỗ này)

Q7. Rò rỉ dữ liệu (data leakage) là gì, sao phải tránh?

- Là khi mô hình "nhìn thấy" thông tin mà lẽ ra lúc dự đoán thật nó không có (ví dụ: thông tin từ tương lai, hoặc thông tin học được từ chính tập dùng để đánh giá).
- Ví von: như học sinh **được xem đáp án trước khi thi** — điểm cao nhưng không phản ánh năng lực thật.
- Hậu quả: điểm số lúc test cao giả tạo, nhưng triển khai thực tế thì tệ hơn hẳn — vì lúc đó không còn "nhìn trộm" được nữa.

Q8. Làm sao đảm bảo đặc trưng lịch sử không bị leakage?

- Nguyên tắc: đặc trưng của lượt đăng nhập tại thời điểm t chỉ được dùng thông tin xảy ra **trước** t của chính user đó.
- Kỹ thuật: `shift(1)` để lấy "lần trước", `cumsum()` - giá trị hiện tại để cộng dồn nhưng loại trừ chính dòng đang xét.
- Bắt buộc `sort_values(["User ID", "Login Timestamp"])` trước, nếu không `shift(1)` sẽ lấy sai dòng.
- ► **Code:** `sort_values` ở `load_raw()`; `shift/cumsum` ở `engineer_features()` — cả hai trong `features.py`.

Q9. Nhóm từng phát hiện lỗi leakage nào, sửa ra sao? (câu này rất nên chủ động nhắc tới — cho thấy tự làm thật)

- **Lỗi 1:** median RTT và tần suất Country/ASN ban đầu tính trên **toàn bộ** dữ liệu (gồm cả val/test) → sửa thành chỉ tính trên **train** (`fit_global_stats()` chỉ nhận `df_train`). ► *Sửa ở:* tách 2 hàm trong `features.py`; gọi đúng cách trong `prepare_splits()` — `dataset_prep.py`; áp lại lúc suy luận ở `predict_risk()` — `inference.py`.
- **Lỗi 2:** ngưỡng min/max của hệ mờ Mamdani từng bị tính lại theo từng lô dữ liệu đưa vào, thay vì cố định từ train → khiến suy luận 1 dòng đơn lẻ bị méo (vì min = max = chính giá trị đó). Sửa: lưu min/max cố định từ lúc `fit()` trên train (`self.minmax_`), dùng lại y hệt cho mọi lần suy luận sau. ► *Sửa ở:* `MamdaniFuzzyRiskSystem.fit()` và hàm `low_med_high()` — đều ở `fuzzy_mamdani.py`.
- Điểm đáng nói: lỗi 2 **không** bộc lộ khi chạy theo lô lớn, chỉ lộ ra khi đưa vào dùng thật → rất dễ bỏ sót.

Q10. Vì sao `user_success_rate_so_far` phải trừ đi chính dòng hiện tại (`cumsum - current`)?

- Nếu không trừ, tỉ lệ thành công "trước đó" sẽ vô tình gồm cả kết quả của **chính lượt đang xét** — tức là dùng thông tin của hiện tại để mô tả quá khứ, một dạng leakage tinh vi dễ bị bỏ sót.
- Trừ đi để đảm bảo con số này chỉ phản ánh đúng những gì đã biết **trước khi** lượt này xảy ra.
- Ví dụ: user có 3 lượt (thành công, thành công, **thất bại**). Ở lượt 3 giá trị đúng là $2/2 = 1,00$; nếu không trừ sẽ thành $2/3 = 0,67$ — đã dùng kết quả lượt 3 để dự đoán lượt 3.
- ► **Code:** dòng `cum_success = success_int.groupby(...).cumsum() - success_int` trong `engineer_features()` — `features.py`.

Nhóm 4 — Chia tập & fit tiền xử lý

Q11. Vì sao chia tập theo kiểu stratified (phân tầng)?

- Ép tỉ lệ ~3% dương phải giữ nguyên ở cả train/val/test. Số thật: 3,0611% / 3,0613% / 3,0613%.
- Nếu chia ngẫu nhiên thường, tập test có thể ngẫu nhiên rơi vào ít mẫu tấn công hơn hẳn → chỉ số đánh giá (đặc biệt AUPRC) dao động mạnh, không đáng tin.
- Quy mô thật: train 350.000 / val 75.000 / test 75.000 dòng.
- ► **Code:** `prepare_splits()` — `dataset_prep.py` (dùng `train_test_split(..., stratify=y)` của `scikit-learn`; tỉ lệ `TEST_SIZE`, `VAL_TEST_SPLIT` ở `config.py`).

Q12. Vì sao mọi thứ "học từ dữ liệu" (scaler, one-hot, ngưỡng mờ, thống kê toàn cục) chỉ được fit trên train?

- Nếu fit trên cả val/test, nghĩa là mô hình (một cách gián tiếp) đã biết trước phân bố của tập dùng để đánh giá nó → điểm số bị thổi phồng, không phản ánh đúng khả năng tổng quát hóa thật.
- Quy tắc chung: train → `fit_transform`; val/test → chỉ `transform`.
- Có 4 bộ phải fit: (1) thống kê toàn cục, (2) hệ mờ Mamdani, (3) `OneHotEncoder`, (4) `StandardScaler`.

- ► **Code:** cả 4 bộ được gọi tập trung trong `prepare_splits()` — `dataset_prep.py`. Bộ (1) là `fit_global_stats()` ở `features.py`; bộ (2) là `MamdaniFuzzyRiskSystem.fit()` ở `fuzzy_mamdani.py`; bộ (3) và (4) là của **scikit-learn**.

Q13. Lúc suy luận, gặp Country/ASN chưa từng thấy trong train thì xử lý sao?

- `country_freq/asn_freq` tra không thấy → NaN → điền 0 (`fillna(0)`).
- `rarity = 1 - freq = 1 - 0 = 1` → tự động thành "rất hiếm" — hợp lý vì địa điểm hoàn toàn lạ thường đáng ngờ hơn.
- Tương tự, thiết bị lạ qua `OneHotEncoder(handle_unknown="ignore")` → toàn bộ cột one-hot = 0, không báo lỗi.
- ► **Code:** xử lý rarity ở `apply_global_stats()` — `features.py`; one-hot ở `prepare_splits()` — `dataset_prep.py`; luồng suy luận ở `predict_risk()` — `inference.py`.

Nhóm 5 — Câu hỏi "bẫy" / đào sâu (kiểm tra hiểu bản chất, không phải học thuộc)

Q14. Nếu 1 user chỉ đăng nhập đúng 1 lần duy nhất trong toàn bộ dữ liệu thì các đặc trưng lịch sử của họ ra sao?

- `is_first_login = 1`.
- `time_since_last_login_h` không có "lần trước" → điền $24 \times 365 = 8760$ giờ (quy ước "rất lâu rồi").
- Tất cả `is_new_country/city/asn/device/browser/os = 1` (mọi thứ đều mới với họ) → `num_changes = 6`.
- `user_success_rate_so_far` không có lịch sử → điền mặc định `1.0`.
- `user_login_count_so_far = 0`.
- → Đáng chú ý: user mới tự động trông giống lượt "nhiều thay đổi" nên dễ bị chấm rủi ro cao — đánh đổi hợp lý, nhưng cũng góp phần vào báo động giả.
- ► **Code:** toàn bộ giá trị mặc định đặt trong `engineer_features()` — `features.py`.

Q15. Vì sao chọn ngưỡng phân vị 20/50/80 cho hệ mờ Mamdani mà không phải con số khác?

- Đây là lựa chọn heuristic hợp lý (chia phân phối thành vùng thấp/giữa/cao tương đối cân bằng), **không phải** kết quả tinh chỉnh (tuning) qua thử nghiệm.
- Nếu cô hỏi sâu hơn: trả lời thẳng đây là hướng có thể cải tiến (vd. để ANFIS tự học ngưỡng thay vì đặt cố định — đã nêu trong phần hướng phát triển).
- ► **Code:** `np.percentile(..., [20, 50, 80])` trong `MamdaniFuzzyRiskSystem.fit()` — `fuzzy_mamdani.py`.

Q16. Input đưa vào MLP có bao nhiêu chiều, tính từ đâu ra?

- **39 chiều** = 19 đặc trưng số đã chuẩn hóa + 4 cột one-hot (**Device Type**: desktop/mobile/tablet/unknown) + 16 đặc trưng mờ Mamdani (5 biến $\times$ 3 mức Low/Med/High = 15, cộng thêm 1 điểm rủi ro centroid).
- ► **Code:** ghép ở cuối `prepare_splits()` — `dataset_prep.py` (lưu dưới khóa `input_dim`); nhận bởi `MLPClassifier.__init__()` — `model.py`.

Q17. Nếu KHÔNG chia stratified mà chia ngẫu nhiên thường thì hậu quả cụ thể là gì?

- Tỷ lệ ~3% dương có thể lệch đáng kể giữa 3 tập (ví dụ test chỉ còn 1–2% hoặc lên tới 5%).
- Với AUPRC — vốn rất nhạy với tỷ lệ lớp dương — số đo được sẽ dao động mạnh giữa các lần chia khác nhau, khó so sánh công bằng và khó tái lập.

- ► **Code:** tham số `stratify=y` trong `prepare_splits()` — `dataset_prep.py`.

Q18. Vì sao phải `sort_values` theo cả `User ID` lẫn `Login Timestamp`, thiếu 1 trong 2 thì sao?

- Thiếu sort theo `User ID`: `groupby("User ID")` vẫn đúng nhóm, nhưng nếu dữ liệu trong nhóm không theo thứ tự thời gian thì `shift(1)` sẽ lấy nhầm dòng không phải "lần đăng nhập ngay trước".
- Thiếu sort theo thời gian: toàn bộ đặc trưng lịch sử (`time_since_last_login_h`, `is_new_*`, `user_success_rate_so_far...`) tính sai hết, vì "quá khứ" và "hiện tại" bị đảo lộn.
- ► **Code:** `load_raw()` — `features.py`.

Q19. Hàm nào trong dự án là nhóm tự viết, hàm nào dùng sẵn của thư viện? (cô rất dễ hỏi câu này)

- **Nhóm tự viết:**
 - `features.py`: `load_raw()`, `engineer_features()`, `fit_global_stats()`, `apply_global_stats()`
 - `fuzzy_mamdani.py`: `triangular()`, `low_med_high()`, lớp `MamdaniFuzzyRiskSystem` (`fit`, `_fuzzify`, `_rule_base`, `transform`)
 - `dataset_prep.py`: `prepare_splits()`
 - `model.py`: `MLPClassifier`, `train_mlp()`, `evaluate()`, `to_tensor()`
 - `inference.py`: `load_pipeline_and_model()`, `predict_risk()` · `visualize.py`: các hàm `plot_*`
- **Dùng sẵn của pandas:** `read_csv`, `to_datetime`, `sort_values`, `groupby`, `shift`, `cumsum`, `cumcount`, `fillna`, `value_counts`.
- **Dùng sẵn của scikit-learn:** `train_test_split`, `StandardScaler`, `OneHotEncoder`, các hàm tính chỉ số (`roc_auc_score`, `average_precision_score`...).
- **Dùng sẵn của PyTorch:** `nn.Linear`, `nn.ReLU`, `nn.BatchNorm1d`, `nn.Dropout`, `BCEWithLogitsLoss`, `Adam`.

Q20. Ngưỡng thật của hệ mờ là bao nhiêu? Có gì bất thường không?

- Ví dụ `time_since_last_login_h`: `p20` = 0,198 giờ (~12 phút), `p50` = 44,6 giờ (~1,9 ngày), `p80` = 360 giờ (15 ngày).
- **Bất thường:** `country_rarity` có `p20` = `p50` = `p80` = **0,1786** — cả ba trùng nhau, vì Na Uy chiếm **82,1%** dữ liệu nên ba phân vị đều rơi vào cùng nhóm này.
- Hệ quả: với biến đó, mức Low và Med **không phân biệt được nhau** (cùng = 1 cho lưu lượng Na Uy), chỉ High là có phân hóa. Code không lỗi (có nhánh xử lý riêng), nhưng độ phân giải của biến bị giới hạn.
- Cách trả lời tốt: thừa nhận hiện tượng → giải thích nguyên nhân → nêu hướng cải tiến (ANFIS tự học ngưỡng).
- ► **Code:** nhánh `if a == b == c` trong hàm `triangular()` — `fuzzy_mamdani.py`. Ngưỡng thật lưu trong pipeline dưới khóa `fis_thresholds` / `fis_minmax`.

Mẹo khi trả lời (nói hoặc viết tay)

1. Ưu tiên trả lời **Q7–Q10 (chống leakage)** kỹ nhất — đây là phần thể hiện tư duy phương pháp luận rõ nhất, và note thuyết trình đã tự nhắc "giám khảo hay hỏi về leakage".
2. **Chủ động nhắc Q9** (hai lỗi tự phát hiện) — cho thấy nhóm tự làm thật và tự kiểm tra lại công việc.
3. **Nói được tên file** khi trả lời (vd "chỗ này em làm ở `features.py`, hàm `engineer_features()`") thì rất thuyết phục — chứng tỏ hiểu code chứ không đọc thuộc. Nếu bí, ít nhất nhớ luồng 4 file: `features.py` → `fuzzy_mamdani.py` → `dataset_prep.py` → `model.py`.
4. Nếu bị hỏi câu không chắc 100%, **trả lời đúng phạm vi mình biết** rồi nói thẳng giới hạn (ví dụ Q15, Q20) — tốt hơn nhiều so với cố nói cho có mà sai.

5. Khi viết tay, ưu tiên gạch đầu dòng ngắn hơn là viết đoạn văn dài — giám khảo chấm nhanh trong thời gian có hạn.
6. **Có số liệu thật thì nêu** (535 ms, 82,1% Na Uy, 39 chiều, 350.000/75.000/75.000) — con số cụ thể luôn thuyết phục hơn nói chung chung.